

GraphOne AI Intelligence Pipeline

Scalable Architecture & Engineering Design

Objective: Build a fault-tolerant, asynchronous intelligence pipeline capable of scaling beyond 500,000 records while maintaining freshness, deduplication, entity consistency and reliable LLM enrichment.

1. End-to-End Data Flow

Sources ? Async Fetchers ? Source Adapters ? Full-Text Extraction ? Date Normalization ? Deduplication ? Entity Resolution ? Intelligent Chunking ? LLM Enrichment ? PostgreSQL/Object Storage ? Graph + Vector Retrieval.

The ingestion layer uses asynchronous HTTP clients and bounded concurrency. Each source has an adapter so that RSS, APIs and HTML pages can be processed independently. Raw snapshots are retained for reproducibility while normalized records are stored separately.

2. 500k+ Scale Strategy

Workers remain stateless and can be horizontally scaled behind a queue. Work is partitioned by source and deterministic record hash. Bounded concurrency, backpressure, batching and checkpointing prevent a large source from exhausting memory or provider quotas. Successful stages are persisted independently so failed workers can resume without repeating completed work.

At production scale, PostgreSQL provides indexed durable metadata, object storage keeps raw documents, and a queue distributes crawl/enrichment jobs. This architecture allows additional workers to be added without changing the source adapters.

3. Async Processing

aiohttp/asyncio provides concurrent network operations. Each source is isolated with timeouts, retry budgets and per-source concurrency limits. Long-running or JavaScript-heavy pages can be delegated to Playwright workers without blocking normal HTTP workers.

Resilience, Freshness & Distributed Processing

4. HTTP 413 Handling

HTTP 413 indicates that the request payload is too large. The pipeline detects payload errors, reduces chunk/request size and retries within a bounded policy. Large documents are split into smaller independently processable chunks while preserving source and entity metadata.

5. HTTP 429 Handling

Rate-limit responses trigger exponential backoff with jitter. Retry-After is respected when supplied. Concurrency is reduced for the affected source and retries are bounded to prevent retry storms.

6. Freshness Validation

Dates are normalized to UTC ISO-8601 using HTML metadata, JSON-LD, time elements and RSS fields such as pubDate, published, updated and dc:date. The freshness validator compares normalized publication time against the current UTC timestamp and accepts records only when they fall inside the required 24-hour window.

7. Distributed Deduplication

Each record receives a deterministic identifier derived from source identity plus canonical URL/content fingerprint. Database unique constraints and idempotent upserts ensure that two workers processing the same article or job cannot create duplicate records. Canonical URL normalization removes tracking parameters and equivalent URL variants where possible.

8. Anti-Bot Strategy

The crawler uses respectful request rates, realistic browser headers, source-specific adapters and RSS/API endpoints whenever available. JavaScript rendering is used only when necessary. The system does not attempt to bypass authentication, CAPTCHAs or access controls. Failures are isolated and retried according to source-specific policies.

Storage, AI & Reasoning Architecture

9. Primary Storage Strategy

SQLite is appropriate for the current local demonstration. For 500k+ production workloads, PostgreSQL is the primary durable store because it provides transactions, indexing, uniqueness constraints and concurrent access. Raw HTML/documents can be stored in object storage, while normalized metadata remains in PostgreSQL.

10. Graph + Vector Reasoning

Entity mappings form a graph-compatible relationship layer connecting startups, products, papers, companies, jobs and sources. Embeddings are stored in a vector index. Each vector chunk retains source ID, entity ID and document metadata, enabling semantic retrieval together with relationship-aware graph reasoning.

11. LLM Fallback Chain

The configured fallback order is Gemini ? Groq ? DeepSeek. Transient failures, quota errors and unavailable responses trigger the next provider. Intelligent chunking limits request size, uses bounded overlap and preserves record metadata so individual chunk failures do not lose the original document.

12. Entity Resolution

Entity resolution combines normalized names, aliases, URLs and deterministic identifiers. Candidate matches are scored before creating mappings. Existing entity IDs are reused whenever confidence is sufficient, preventing duplicate representations of the same organization, product or research entity.

13. Observability & Recovery

The pipeline records HTTP status, latency, retry count, freshness, extraction success, deduplication results and LLM provider usage. Checkpoints and idempotent writes allow recovery from worker crashes. Failed records can be routed to a dead-letter workflow without stopping the complete ingestion pipeline.

Conclusion: The architecture separates crawling, extraction, enrichment and storage so each layer can scale independently. Stateless async workers, deterministic identity, freshness validation, provider fallback and durable storage provide the foundation required for a 500k+ record intelligence platform.